5 - 任务调度

本节重点

掌握 Spring Scheduler 定时任务的用法，并且学习动态多定时任务的设计实现。

内容包括：

- 定时任务技术选型
- Spring Scheduler 实战
- 多任务并发调度
- 动态任务配置与管理

一、定时任务技术选型

除了我们项目选用的 Spring Scheduler 之外，还有很多任务调度技术，比如 Quartz、分布式任务调度平台 XXL-JOB 等。每种技术都有自己的特点和适用场景，了解它们有助于我们在实际项目中做出更好的选择。

Spring Scheduler

Spring Scheduler 最大的优点就是简单。不需要额外配置什么中间件，直接在代码里用注解就能实现定时任务。对于大多数单体应用来说，这已经完全够用了。

不过它也有局限性，比如不支持分布式环境，也没有可视化的管理界面。如果你的应用只部署在一台服务器上，任务也不是特别复杂，那 Spring Scheduler 是首选。

Quartz

Quartz 是老牌的 Java 定时任务框架，功能非常强大。它支持各种复杂的调度策略，可以把任务信息存储到数据库中，还能暂停、恢复任务等等。很多企业级应用都在使用 Quartz。

但是 Quartz 的分布式能力比较弱，主要是通过数据库锁来协调多个节点，在高并发场景下可能会成为瓶颈。而且它没有现成的管理界面，需要自己开发。

XXL-JOB 分布式调度

XXL-JOB 是国内开发的分布式任务调度平台，在国内企业中非常受欢迎。它最吸引人的地方就是提供了一个很好用的 Web 管理界面，你可以在网页上直接添加、修改、删除任务，

还能查看任务的执行日志。

XXL-JOB 天然支持分布式部署，可以把任务分发到不同的服务器上执行，还支持故障转移。如果你的系统是微服务架构，或者需要在多台服务器上运行任务，XXL-JOB 是个很好的选择。

💡 编程导航的代码生成器共享平台项目 第 14 期有关于 XXL-JOB 分布式定时任务的讲解，推荐学习。

其他

除了这些，还有 Elastic-Job、PowerJob 等框架。Elastic-Job 基于 ZooKeeper，支持任务分片，适合处理大量数据的场景。PowerJob 是比较新的框架，性能很不错，支持复杂的工作流，但学习成本稍高一些。

如何选择？

选择哪种技术主要看你的项目需求：

- 如果是简单的单体应用，任务不多不复杂，直接用 Spring Scheduler 就行了，省心省力。
- 如果是分布式系统，或者需要可视化管理任务，XXL-JOB 是个不错的选择，上手也不难。
- 如果对性能要求特别高，或者需要处理复杂的任务依赖关系，可以考虑 PowerJob 或者 Elastic-Job。

为什么我们选择 Spring Scheduler？

回到我们的 AI 自动回复工具，选择 Spring Scheduler 主要是因为项目相对简单，就是定期去监控知识星球有没有新问题，然后调用 OpenAI 回答。这种场景用 Spring Scheduler 完全够用，而且部署简单，不需要额外搭建什么服务。

当然，如果以后项目变复杂了，比如要支持监控多个平台，或者需要更精细的任务管理，我们完全可以升级到 XXL-JOB。技术选型本来就是一个渐进的过程，从简单开始，根据需求逐步演进。

二、Spring Scheduler 实战

Spring Framework 提供了强大的任务调度功能，通过 `@Scheduled` 注解和 `SchedulingConfigurer` 接口，我们可以实现灵活的定时任务调度。

在我们的项目中，任务调度是整个系统的核心，它负责按照配置的周期执行监控和回答任务。

启用定时任务

首先需要在启动类中启用 Spring 的定时任务功能：

▼
java
复制代码
```
@SpringBootApplication
@EnableScheduling  // 启用定时任务
@EnableAspectJAutoProxy(proxyTargetClass = true, exposeProxy = true)
@Slf4j
public class MainApplication implements CommandLineRunner {

    public static void main(String[] args) {
        SpringApplication.run(MainApplication.class, args);
    }

    @Override
    public void run(String... args) throws Exception {
        // 启动时输出配置信息
        OpenAiConfig openAiConfig = SpringContextUtils.getBean(OpenAiConfig.class);
        ZsxqConfig zsxqConfig = SpringContextUtils.getBean(ZsxqConfig.class);
        log.info("OpenAi 配置 {}", zsxqConfig);
        log.info("知识星球配置 {}", openAiConfig);
    }
}
```

`@EnableScheduling` 注解是启用定时任务的关键，它会自动配置 Spring 的任务调度器。

任务配置类

项目采用了 `SchedulingConfigurer` 接口来实现动态任务配置，这比使用 `@Scheduled` 注解更加灵活：

▼
java
复制代码
```
@Configuration
@ConfigurationProperties(prefix = "task")
```

```java
@Data
@Slf4j
public class TaskConfig implements SchedulingConfigurer {

    /**
     * 并发配置
     */
    private ConcurrentConfig concurrent = new ConcurrentConfig();

    /**
     * 任务列表
     */
    private List<TaskListItem> list;

    @Override
    public void configureTasks(ScheduledTaskRegistrar taskRegistrar) {
        // 配置线程池（支持并发执行）
        if (BooleanUtils.isTrue(concurrent.getEnable())) {
            // 如果开启并发，默认并发度为任务数，即全并发，可通过配置更改
            int size = Optional.ofNullable(concurrent.getSize()).orElse(list.size());
            taskRegistrar.setScheduler(Executors.newScheduledThreadPool(size));
        }

        // 动态注册任务
        log.info("--- 任务注册开始 ---");
        for (int i = 0; i < list.size(); i++) {
            TaskListItem taskListItem = list.get(i);
            if (StringUtils.isBlank(taskListItem.getName())) {
                taskListItem.setName("task" + (i + 1));
            }
            // 注册 Cron 任务
            taskRegistrar.addCronTask(new JobMediator(taskListItem), taskListItem.getCron());
            log.info("任务注册成功 {}", taskListItem);
        }
        log.info("--- 任务注册结束 ---");
    }

    @Data
    public static class ConcurrentConfig {
        private Boolean enable = false;
        private Integer size;
    }
}
```

这段代码中的几个实现关键：

1. 动态配置：通过 `@ConfigurationProperties` 从配置文件读取任务列表
2. 灵活并发：支持串行或并行执行，可配置线程池大小
3. 任务隔离：每个任务独立运行，互不影响
4. 自动命名：自动为未命名的任务生成默认名称

任务配置模型

支持配置多个不同的任务，每个任务可以有独立的执行周期、监控源（知识星球 / 默认等）、回答源（OpenAI / 默认等）。

▼

java
复制代码

```java
@Data
public class TaskListItem {
    /**
     * 任务名
     */
```

```
    private String name = "";

    /**
     * 任务执行周期（Cron 表达式）
     */
    private String cron = "0/30 * * * * ?";

    /**
     * 回答者类型
     */
    private String answerer = "openai";

    /**
     * 监控者类型
     */
    private String monitor = "zsxq";
}
```

Cron 表达式详解

Cron 表达式是定时任务的核心，理解它的格式对于配置任务非常重要。

▼
plain
复制代码

```
秒 分 时 日 月 周 [年]
*  *  *  *  *  *   *
```

举一些例子：

▼
yaml
复制代码

```
# 每 30 秒执行一次
cron: '0/30 * * * * ?'

# 每分钟执行一次
cron: '0 * * * * ?'

# 每小时执行一次
cron: '0 0 * * * ?'

# 每天上午 9 点执行
cron: '0 0 9 * * ?'

# 工作日上午 9 点执行
cron: '0 0 9 * * MON-FRI'

# 每月 1 号上午 9 点执行
cron: '0 0 9 1 * ?'
```

不过网上有很多可视化 Cron 表达式生成器，再加上有了 AI，其实没必要手写了。

三、多任务并发调度

并发执行配置

项目支持两种执行模式：

1. 串行执行（默认）

▼
yaml
复制代码
```yaml
task:
  concurrent:
    enable: false  # 关闭并发
  list:
    - name: task1
      monitor: zsxq
      answerer: openai
      cron: '0/30 * * * * ?'
    - name: task2
      monitor: default
      answerer: default
      cron: '0/10 * * * * ?'
```

串行模式下，任务按顺序执行，不会出现资源竞争，适合对资源敏感的场景。

2、并发执行
▼
yaml
复制代码
```yaml
task:
  concurrent:
    enable: true   # 启用并发
    size: 2        # 线程池大小
  list:
    - name: task1
```

```
      monitor: zsxq
      answerer: openai
      cron: '0/30 * * * * ?'
    - name: task2
      monitor: default
      answerer: default
      cron: '0/10 * * * * ?'
```

并发模式下，多个任务可以同时执行，提高系统吞吐量。

线程池配置策略

可以给定时任务注册器设置线程池，实现任务的并发执行，默认并发度为任务数，即全并发，也可以通过配置更改。

▼

java
复制代码

```
if (BooleanUtils.isTrue(concurrent.getEnable())) {
    int size = Optional.ofNullable(concurrent.getSize()).orElse(list.size());
    taskRegistrar.setScheduler(Executors.newScheduledThreadPool(size));
}
```

任务执行流程

项目中的任务执行流程通过 JobMediator 中介者模式实现：

▼

java
复制代码

```
public class JobMediator implements Runnable {

    private final TaskListItem taskListItem;

    public JobMediator(TaskListItem taskListItem) {
        this.taskListItem = taskListItem;
    }

    @Override
    public void run() {
        // 根据配置选择 monitor 和 answerer
        Monitor monitor = MonitorFactory.createMonitor(taskListItem.getMonitor(), taskListItem);
        Answerer answerer = AnswererFactory.createAnswerer(taskListItem.getAnswerer());
        // 监控并回答
        monitor.onMonitor(answerer);
    }
}
```

具体的执行流程：

1. 任务创建：每个任务都被包装成 JobMediator 实例
2. 组件获取：通过工厂模式动态创建监控者和回答者
3. 业务执行：调用监控者的 onMonitor 方法开始执行
4. 异常隔离：单个任务异常不会影响其他任务

四、动态任务配置与管理

配置文件管理

项目支持在 `application.yml` 中灵活配置多个任务：

▼

yaml
复制代码

```yaml
task:
  concurrent:
    enable: false    # 并发开关
    size: 1         # 线程池大小
  list:
    - name: task1
      monitor: zsxq
      answerer: openai
      cron: '0/30 * * * * ?'
    # 可以继续添加更多任务
    # - name: task2
    #   monitor: default
    #   answerer: default
    #   cron: '0/10 * * * * ?'
```

任务生命周期管理

1、任务注册阶段

▼

java
复制代码

```java
log.info("--- 任务注册开始 ---");
for (int i = 0; i < list.size(); i++) {
    TaskListItem taskListItem = list.get(i);
    if (StringUtils.isBlank(taskListItem.getName())) {
        taskListItem.setName("task" + (i + 1));
    }
    taskRegistrar.addCronTask(new JobMediator(taskListItem), taskListItem.getCron());
    log.info("任务注册成功 {}", taskListItem);
}
log.info("--- 任务注册结束 ---");
```

2、任务执行监控

每个任务都会记录详细的执行日志，便于监控和调试，包括：

- 任务开始和结束时间
- 处理的问题数量
- 成功和失败的回答数
- 异常信息记录

任务配置最佳实践

1、合理设置执行频率

▼

yaml
复制代码

```yaml
# 生产环境建议设置较长间隔，避免频繁请求
cron: '0/30 * * * * ?'  # 30秒间隔
# 测试环境可以设置短间隔
cron: '0/10 * * * * ?'  # 10秒间隔
```

2、监控任务健康状态

▼

java
复制代码

```java
// 在 Monitor 实现中添加健康检查
public void onMonitor(Answerer answerer) {
    String taskName = taskListItem.getName();
    log.info("任务 {} 监控开始", taskName);
    try {
        // 执行监控逻辑
        // ...
        log.info("任务 {} 监控结束", taskName);
    } catch (Exception e) {
        log.error("任务 {} 执行异常", taskName, e);
        // 可以添加告警逻辑
    }
}
```

3、环境变量支持

▼

yaml
复制代码

```yaml
task:
  list:
    - name: ${TASK_NAME:task1}
      monitor: ${MONITOR_TYPE:zsxq}
      answerer: ${ANSWERER_TYPE:openai}
      cron: ${TASK_CRON:0/30 * * * * ?}
```

4、区分不同环境配置

比如开发环境使用串行，便于调试：

▼

yaml
复制代码

```yaml
task:
  concurrent:
    enable: false  # 开发时使用串行，便于调试
  list:
    - name: dev_task
      monitor: default
      answerer: default
      cron: '0/5 * * * * ?'  # 5秒间隔，快速验证
```

生产环境启动并发，而且需要合理调整定时任务间隔：

▼

yaml
复制代码

```yaml
task:
  concurrent:
    enable: true   # 生产环境启用并发
    size: 2
  list:
    - name: prod_task
      monitor: zsxq
      answerer: openai
      cron: '0/30 * * * * ?'  # 30秒间隔，避免频繁请求
```

五、扩展思路

1）任务状态管理

- 添加任务启用/禁用开关
- 实现任务暂停和恢复功能
- 记录任务执行历史

2）动态配置更新：支持运行时修改任务配置，可以利用 Nacos 实现配置热更新机制，并且添加配置变更通知。编程导航的智能面试刷题平台项目 有基于 Nacos 实现动态配置的实战，推荐学习。

3）任务监控增强

- 添加任务执行时长统计
- 实现任务执行失败重试机制
- 提供任务性能指标监控

4）分布式任务调度

- 支持多实例部署
- 实现任务分片执行
- 添加任务负载均衡